Full-Stack Developer Assignment

Three-Way Match Engine for PO, GRN, and Invoice

Build a full-stack application that lets users upload **Purchase Order (PO)**, **Goods Receipt Note (GRN)**, and **Invoice** documents, extracts structured data using the **Gemini API**, resolves each line item against a **SKU Master catalogue**, stores everything in **MongoDB**, and performs a **three-way match** — surfaced through a UI that follows the reference screenshots at the end of this document.

This is a backend + frontend problem-solving exercise, not a polished production build. Favor a clean, sensible implementation over pixel-perfect chrome, and use your judgement to fill in any details this document doesn't spell out.

Timeline: 5–6 days.

Overview — What You're Building

In procurement, the same purchase is described by three different documents: the **PO** (what was ordered), the **GRN** (what was actually received at the warehouse), and the **Invoice** (what the vendor is billing for). A three-way match is the reconciliation step that answers: *do these three agree?* Your job is to automate it end to end.

The end-to-end flow you need to deliver:

1. A user logs in and uploads a document (PDF or image), tagging it as PO, GRN, or Invoice.
2. The backend stores the raw file and sends it to Gemini with a document-type-specific prompt, which returns structured JSON (header fields + line items).
3. Each extracted line item is **resolved against the SKU Master** — the vendor's item code on the document is looked up so that the same physical product can be recognised across all three documents even when the text on each differs.
4. The document is persisted, keyed by its `poNumber`, and a duplication check runs.
5. Whenever the match is read, the engine recomputes item-level quantity/price/MRP comparisons across all currently stored documents and rolls them up into a single PO-level status with reason codes.
6. The frontend shows the PO and its linked GRNs and Invoices across tabs, with the original file preview, an item grid, and mismatched cells highlighted.

Two things carry most of the weight in review:

- **Master resolution is what makes matching possible.** A PO line may read `BIK-BIKANERI-200G` while the GRN for the same product reads `Bikaji Bikaneri Bhujia 200 G Pp`. Matching on raw strings fails; matching on the resolved SKU Master record works. Items that can't be resolved must stay visible as warnings, never be silently dropped.
- **Upload order must not matter.** An Invoice may arrive before the PO exists. Documents are

linked by the `poNumber` *string*, not by a foreign key to an existing PO record, so every document is storable on its own and the match result is always derived from whatever is currently in the database.

Required Stack

Backend — Node.js, Express, MongoDB (Mongoose or native driver), Gemini API for parsing, any supporting libraries you need (upload middleware, validation, etc.).

Frontend — Next.js (**App Router**), Tailwind CSS, and one state-management approach of your choice: **Redux Toolkit** (single global store, explicit actions/reducers) or **TanStack Query** (backend as source of truth, caching/invalidation, small local store/context for UI state). Justify your pick briefly in the README.

Auth — no real identity provider needed. A `POST /auth/login` that returns a static Bearer token is enough; the frontend stores it and attaches it to all protected requests.

Data Model

SkuMaster — `skuErpCode` (string, unique — the code that appears on PO/GRN/Invoice lines), `name`, `eanCode` (alternate lookup key), `hsnCode`, `uom`, `agreedRate` (contracted unit price), `mrp`, `priceTolerance` (fraction, e.g. `0.05` = 5%)

PurchaseOrder — `poNumber` (unique), `poDate`, `vendorName`, `items[]` (`itemCode`, `description`, `quantity`, `skuMaster` ref nullable), `rawParsed`, `createdAt`

Grn — `grnNumber` (unique per `poNumber`), `poNumber` (link key — **PO need not exist yet**), `grnDate`, `items[]` (`itemCode`, `description`, `receivedQuantity`, `mrp`, `skuMaster` nullable), `rawParsed`

Invoice — `invoiceNumber` (unique per `poNumber`), `poNumber`, `invoiceDate`, `items[]` (`itemCode`, `description`, `quantity`, `unitRate`, `mrp`, `skuMaster` nullable), `rawParsed`

MatchAudit — `poNumber`, `steps[]` (`{ step, status, message, at }`) — a simple log, one entry per upload event.

Keep `rawParsed` around: it's the unmodified Gemini output, useful for debugging extraction problems without re-uploading. ERP and EAN codes are strings, not numbers.

Item matching key — use the resolved `SkuMaster._id` across PO/GRN/Invoice. If an item can't be resolved, fall back to the normalised raw `itemCode` and flag `unmapped_master_sku` rather than dropping it. Explain this choice (or your own) in the README.

Parsing, Master Resolution & Duplication

Parsing — receive `file` + `documentType` (`po` / `grn` / `invoice`), store the raw file (local disk is fine), call Gemini with a document-type-specific prompt, validate the returned JSON has the minimum required fields (retry once on malformed output, then fail clearly), and persist the parsed

document **independently of whether a PO already exists for that** `poNumber`. Treat Gemini output as untrusted input — validate before persisting, and don't store partial or malformed documents silently.

Minimum fields to extract — **PO**: `poNumber`, `poDate`, `vendorName`, `items[]` (`itemCode`, `description`, `quantity`). **GRN**: `grnNumber`, `poNumber`, `grnDate`, `items[]` (`itemCode`, `description`, `receivedQuantity`). **Invoice**: `invoiceNumber`, `poNumber`, `invoiceDate`, `items[]` (`itemCode`, `description`, `quantity`, and `unitRate` / `mrp` where visible on the document). Use the supplied sample files to decide anything beyond this.

Master resolution (run after parsing, before matching) — look up `SkuMaster` by `skuErpCode == itemCode`; if not found, try `eanCode == itemCode`; if still not found, leave `skuMaster` unset and record `unmapped_master_sku` as a soft warning. Comparison should at least trim whitespace and be case-insensitive. Never block storage of a document for this reason — and if the missing SKU Master record is created later, a recomputed match should pick it up.

Duplication check (run right after persistence) — a second PO for a `poNumber` that already has one → `duplicate_po` (store it anyway, don't overwrite, surface the conflict). A second GRN/Invoice reusing a `grnNumber` / `invoiceNumber` under the same `poNumber` → `duplicate_document`.

Implement the pipeline as plain functions called in sequence — no engine/plugin abstraction is expected for a 5–6 day assignment.

Matching Rules

Validate at **item level**, then aggregate to a PO-level status. Where the same SKU appears on multiple lines of a document, aggregate its quantities. Assume comparable units across documents — UOM conversion is out of scope.

Reason code	Rule
`grn_qty_exceeds_po_qty`	Total received qty (all GRNs) must not exceed PO qty
`invoice_qty_exceeds_grn_qty`	Total invoiced qty (all Invoices) must not exceed total GRN qty
`invoice_qty_exceeds_po_qty`	Total invoiced qty must not exceed PO qty
`invoice_date_after_po_date`	No Invoice date may be after the PO date
`duplicate_po`	A second PO was uploaded for a `poNumber` that already has one
`item_missing_in_po`	Item on a GRN/Invoice has no corresponding PO item (by `SkuMaster`, or raw `itemCode` if unresolved)
`price_mismatch`	Invoice `unitRate` differs from `SkuMaster.agreedRate` by more than `priceTolerance`
`mrp_mismatch`	Invoice/GRN `mrp` differs from `SkuMaster.mrp` by more than ~1%
`unmapped_master_sku`	Item could not be resolved to any `SkuMaster` record

Missing rates or MRPs should not by themselves produce a mismatch, and a zero/invalid agreed rate must not cause a divide-by-zero. You may add codes; these are the minimum.

Status (`GET /match/:poNumber`): `insufficient_documents` (the full PO + GRN + Invoice set isn't available

yet — missing types are not treated as zero quantities) → `mismatch` (any hard violation: `*_qty_exceeds_*`, `invoice_date_after_po_date`, `duplicate_po`, `duplicate_document`, `item_missing_in_po`) → `partially_matched` (no hard violations, but quantities not fully reconciled or soft warnings exist: `price_mismatch`, `mrp_mismatch`, `unmapped_master_sku`) → `matched` (fully reconciled across all three, no reasons at all).

`GET /match/:poNumber` must always **recompute from current stored documents**, never return a stale cached result.

Minimum APIs

```
POST  /auth/login                              → { token }


POST  /documents/upload   (multipart: file, documentType)
GET   /documents/:id
GET   /documents/:id/file                      → original file for preview
GET   /documents?type=&poNumber=


GET   /match/:poNumber      → status, overall + item-level reasons, linked docs,
                              per-item PO/GRN/Invoice totals, resolved SKU info
GET   /summary/:poNumber    → see Summary tab below


POST/GET/PATCH/DELETE  /masters/sku[/:id]
```

All routes except `/auth/login` require `Authorization: Bearer <token>`. Return sensible HTTP status codes and clear error messages; never leak stack traces or API keys.

Frontend Specification

Build a UI that **reads like** the reference screenshots below — same tabs, same panel layout, same information density. It doesn't need to be pixel-identical.

> **Naming note:** tab labels intentionally don't match document-type names — **`Fulfillment` shows Invoices**, **`Delivery` shows GRNs**. Keep these exact labels.

- **App shell**: left icon rail (static is fine), top tabs — `Purchase Order (1)`, `Fulfillment (N)`, `Delivery (N)`, `Summary` — each with a count badge. Within Fulfillment/Delivery, one sub-tab pill per document (e.g. `GRN: 5107297866 Raised`) so the user can switch between multiple GRNs/Invoices.
- **Document detail view** (PO / Fulfillment / Delivery tabs): left column — bordered form sections with a colored left-accent bar and read-only labeled fields (GRN/Invoice/PO details as relevant), plus a mismatch banner above the form when applicable (e.g. "Price Mismatch"). Right column — preview of the original PDF/image with basic zoom (`-`, `%`, `+`); a plain `<iframe>` / `<img>` is enough, and a clear fallback message when preview isn't possible. Bottom — full-width item grid

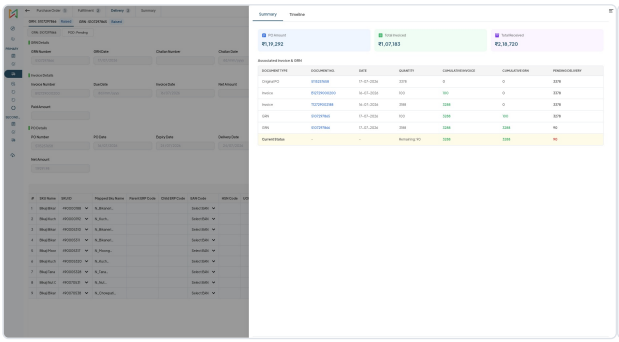
(SKU Name, SKU ID, Mapped SKU Name, ERP Code, EAN, HSN, UOM, quantity columns, Unit Price, Unit MRP, Gross Amount) with mismatched price/MRP cells highlighted and unmapped items visibly flagged.

- **Summary tab**: three stat cards (PO Amount, Total Invoiced, Total Received) and an "Associated Invoice & GRN" table — one row per document plus a final Current Status row, showing cumulative invoiced/received quantities and pending delivery. This is exactly what `GET /summary/:poNumber` should return; the frontend just renders it.
- **Upload**: modal/page to pick `documentType` + file, submit to `/documents/upload`, show progress and any parsing/validation errors, and refresh the match result on success.
- **SKU Master**: list, create, edit, delete screens with basic validation and backend error display.
- **Auth**: login screen that stores the token and attaches it via a fetch wrapper/interceptor; unauthenticated users shouldn't reach protected screens.

Reference Screenshots

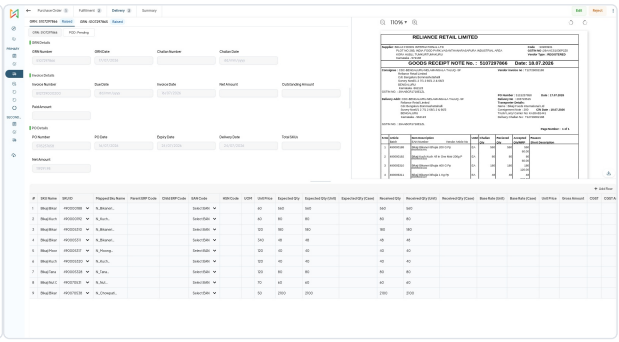
Summary tab

Stat cards + Associated Invoice & GRN cumulative table



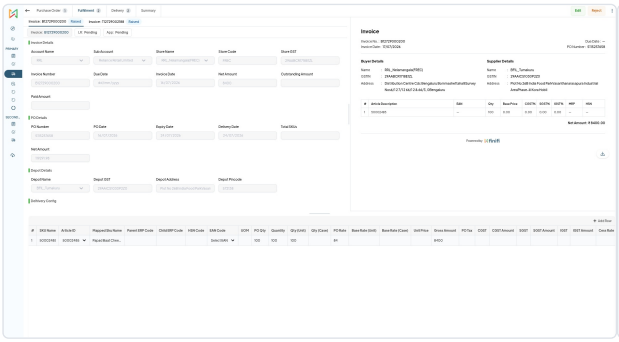
Delivery (GRN) detail view

Form panel + file preview + item grid



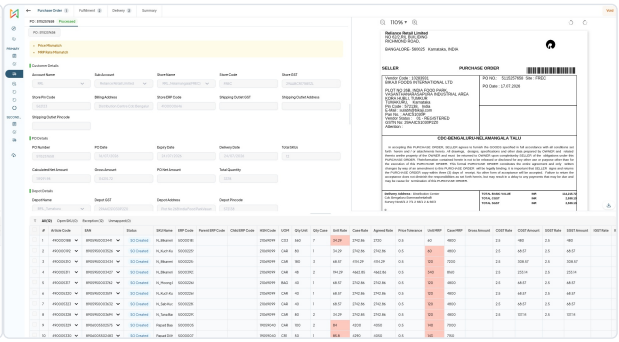
Fulfillment (Invoice) detail view

Form panel + file preview



Purchase Order detail view

Mismatch banner + item grid with highlighted cells



Deliverables & README

Public GitHub repo containing: working backend + frontend (both runnable locally), an `.env.example` (no committed secrets), a `README.md` (setup/run steps, Gemini config, approach, data model, parsing flow, matching-key rationale, matching logic, out-of-order handling, duplicate handling, frontend architecture + state-management choice, assumptions, tradeoffs, known limitations, what you'd improve, and which AI tools you used), API docs (Postman or Swagger/OpenAPI), sample outputs (parsed JSON, a `GET /match/:poNumber` result, a `GET /summary/:poNumber` result), and screenshots of your own working UI.

Evaluated on: matching correctness, master resolution correctness, out-of-order robustness, duplicate handling, API and error-handling quality, code simplicity and readability, UI fidelity to the reference screens, and README clarity. You're free to use AI assistants, but you must be able to explain, debug, and modify everything you submit — the review may include changing one of the matching rules live.

Bonus (optional)

- Swagger/OpenAPI docs or a Postman collection.
- Closer visual fidelity to the reference screenshots (badges, section styling, mismatch banners, highlighted cells, responsive behaviour).
- Real (non-visual) upload progress — `uploading → parsing → mapping → matched` reflecting actual backend state.

Assumptions

Local disk storage is fine (no cloud blob storage). Mock auth (static token) is sufficient. `SkuMaster` resolution is the suggested item-matching key, but you may propose your own — just explain the choice in the README.